

Handbuch: Python Tool-Server API (Core-Module)

Dieses Handbuch beschreibt die **Core-Module** des lokalen Python-Tool-Servers und deren erwartete Request/Response-Strukturen. Die eigentlichen HTTP-Routen liegen typischerweise in `python_server.py` (nicht in den hochgeladenen Dateien enthalten). Die hier dokumentierten Endpunkte sind daher als **empfohlene/uebliche Mapping-Namen** zu verstehen - die JSON-Payloads und Rueckgaben entsprechen jedoch exakt den Core-Funktionen.

Dokumentierte Module:

- `service_core.py` - Start/Stop/Status/Restart eines lokalen Service-Prozesses
- `upload_core.py` - Dateiauswahl + Kopie nach Webroot (`/pickUpload`)
- `filepicker_core.py` - OS-Dateidialog (File/Folder Picker)
- `open_folder.py` - Ordner im nativen Dateimanager oeffnen
- `printer_core.py` - Druckerlisten, Default setzen, GUI oeffnen
- `close_browser.py` - Browser-Prozess anhand der HTTP-Connection finden und beenden
- `render_core.py` - Collage-Renderer (Template XML + Photos + Greenwall)

1) Authentifizierung (api_key / AuthKey)

Mehrere Module sind fuer einen **lokalen, abgesicherten Betrieb** gedacht. In `service_core.py` ist eine einfache Key-Pruefung implementiert:

- Erwarteter Key steht in `server_config.json` unter `AuthKey`.
- Der Server sollte den vom Client gelieferten Key (typisch Query-Parameter `api_key`) gegen `AuthKey` pruefen.
- Vergleich erfolgt timing-safe via `hmac.compare_digest`.

Fehlercodes aus `validate_api_key()`:

- `api_key_not_configured`
- `api_key_missing`
- `api_key_mismatch`

2) Konfiguration

2.1 server_config.json

`service_core.py` erwartet eine Konfigurationsdatei neben `python_server.py`. Minimalbeispiel:

```
{
  "AuthKey": "DEIN_KEY",
  "Port": 8051,
  "args": ["--one-instance", "--port", "{port}"]
}
```

```
}
```

Wichtige Keys:

- `AuthKey` (string) - Shared Secret fuer `api_key`.
- `Port` (int) - Port, der in `{port}`-Platzhalter eingesetzt wird.
- `args` (list oder string) - Startparameter fuer den Service-Prozess. `{port}` wird automatisch ersetzt.
- `caddyWebroot` (string) - wird von `upload_core.py` verwendet (siehe unten).

2.2 caddyWebroot (Upload-Ziel)

`upload_core.py` sucht den Webroot in mehreren Varianten:

- `caddyWebroot`
- `paths.caddyWebroot`
- `caddy.webroot`
- `webroot`

Wenn der Pfad relativ ist, wird er relativ zum Server-Verzeichnis aufgeloeset.

2.3 render_config.json

`render_core.py` laedt standardmaessig `config/render_config.json` relativ zu einem "booth root" (Parent-Ordner, der ein `config/` Verzeichnis enthaelt). Wenn keine Datei existiert, werden Defaults genutzt.

Optional kann ein expliziter Pfad via Payload `render_config` uebergeben werden.

3) Response-Konventionen

Fast alle Core-Funktionen geben ein JSON-Objekt (Python dict) mit diesen Konventionen zurueck:

- `ok`: `true|false` - Erfolg.
- `error`: `<code>` - kurzer Fehlercode (bei `ok=false`).
- Viele Antworten enthalten zusaetzlich `http_status` als Empfehlung fuer den HTTP-Status.
- Wichtig: Bei User-Cancel im Filepicker ist `ok=false`, aber `http_status` bleibt `**200**`, weil es kein technischer Fehler ist.

4) Service-Management (`service_core.py`)

Ziel: Einen lokalen Prozess (z.B. `ApiServer.exe`) starten, stoppen, Status lesen, neu starten.

OS-Support:

- Windows: implementiert.

- Linux/macOS: aktuell Platzhalter (not_implemented_on_this_os).

4.1 Status

Core-Funktion:

- `get_service_status(exe_path: str) -> dict`

Rueckgabe (Erfolg):

```
{
  "ok": true,
  "running": true,
  "pids": [1234],
  "exe": "C:/path/ApiServer.exe",
  "exeName": "ApiServer.exe",
  "detail": {"items": [{"pid": 1234, "path": "C:/path/ApiServer.exe", "name":
    "ApiServer.exe"}]}
```

Fehlercodes:

- `not_implemented_on_this_os`
- `unsupported_os`
- `process_query_failed`

Windows-Details:

- Primaer: PowerShell/CIM (Get-CimInstance Win32_Process) liefert PID + ExecutablePath.
- Fallback: `tasklist` (liefert nur PID, kein Path).

4.2 Start

Core-Funktion:

- `start_service(exe_path: str, cfg: dict) -> dict`

Rueckgabe (bereits aktiv):

```
{ "ok": true, "alreadyRunning": true, "status": {"ok": true, "running": true,
  "pids": [1234]} }
```

Rueckgabe (gestartet):

```
{
  "ok": true,
  "started": true,
  "pid": 5678,
  "cmd": ["C:/path/ApiServer.exe", "--one-instance", "--port", "8051"],
  "status": {"ok": true, "running": true, "pids": [5678]}
```

Fehlercodes:

- `exe_not_found`
- `start_failed`
- `not_implemented_on_this_os`

Hinweise:

- Windows-Start erfolgt detached (ohne Konsolenfenster).
- Danach erfolgt ein kurzer Status-Recheck.

4.3 Stop

Core-Funktion:

- `stop_service(exe_path: str) -> dict`

Rueckgabe (bereits gestoppt):

```
{ "ok": true, "alreadyStopped": true, "status": {"ok": true, "running": false,
  "pids": []} }
```

Rueckgabe (kill by PID):

```
{
  "ok": true,
  "killed": [
    {"pid": 1234, "rc": 0, "out": "...", "err": ""}
  ],
  "status": {"ok": true, "running": false, "pids": []}
}
```

Fallback (kein PID gefunden): kill by ImageName:

```
{
  "ok": true,
  "killedBy": "image",
  "cmd": ["cmd", "/c", "taskkill", "/IM", "ApiServer.exe", "/F", "/T"],
  "rc": 0,
  "out": "...",
  "err": "...",
  "status": {"ok": true, "running": false, "pids": []}
}
```

4.4 Restart

Core-Funktion:

- `restart_service(exe_path: str, cfg: dict) -> dict`

Rueckgabe:

```
{ "ok": true, "stop": { ... }, "start": { ... } }
```

5) Upload via Picker (`upload_core.py`) - `/pickUpload`

Ziel: Ein Filepicker wird auf dem Server-Host geoeffnet, der User waehlt eine Datei aus. Diese wird nach `<webroot>/uploads[/subdir]` kopiert und Metadaten werden zurueckgegeben.

5.1 Query-Parameter

`pick_upload()` verarbeitet (typisch als Query):

- `title` (string, default: Select) - Dialogtitel.
- `path` (string, default: leer) - initialer Pfad fuer den Picker.
- `filter` (string, default: Images|*.png;*.jpg;*.jpeg;*.webp;*.bmp;*.gif|All|*.*)
- `subdir` (string, default: leer) - Unterordner unter `uploads/`.
- `prefix` (string, default: leer) - Dateipraefix.
- `overwrite` (bool-ish, default: false) - wenn false: `_1`, `_2`, ... wird angehaengt.

5.2 subdir-Sanitizing (Traversal-Schutz)

`subdir` wird stark eingeschaenkt:

- Nur A-Z a-z 0-9 . _ - /.
- Keine fuehrenden Slashes.
- . und .. werden entfernt.
- Jede Path-Komponente muss `^[A-Za-z0-9._-]+$` matchen.

5.3 Filter-Format (Windows vs Linux)

- Windows-Style: Images|*.png;*.jpg;*.jpeg|All|*.*
- Linux (zenity) erwartet: Images | *.png *.jpg *.jpeg

`normalize_filter()` konvertiert auf Linux automatisch das erste Filterpaar und ersetzt ; durch Leerzeichen.

5.4 Rueckgabe (Erfolg)

```
{
  "ok": true,
  "webroot": "C:/.../webroot",
  "uploads_dir": "C:/.../webroot/uploads/event1",
  "subdir": "event1",
  "source_abs": "C:/Users/.../Pictures/img.jpg",
  "saved_abs": "C:/.../webroot/uploads/event1/prefix_img.jpg",
  "saved_rel": "uploads/event1/prefix_img.jpg",
  "saved_url": "/uploads/event1/prefix_img.jpg",
  "file_name": "prefix_img.jpg",
  "size_bytes": 123456,
  "mime": "image/jpeg",
  "overwrite": false,
  "prefix": "prefix",
  "modified_at": "2026-01-16T10:04:00",
}
```

```
"created_at": "2026-01-16T10:03:50",  
"http_status": 200  
}
```

5.5 Cancel / Fehler

- User cancel: ok=false, picked="", http_status=200.
- Technische Fehler:
- missing_or_invalid_caddyWebroot (500)
- uploads_dir_create_failed (500)
- picker_timeout (500)
- picker_failed (500)
- picked_not_a_file (400)
- copy_failed (500)

6) File/Folder Picker (`filepicker\_core.py`)

Dieses Skript ist ein **CLI-Wrapper** fuer native Dialoge.

CLI-Parameter:

- --mode file|folder
- --title <text>
- --path <initial path>
- --filter <filter string> (nur file-mode)

OS-Support:

- Windows: COM IFileOpenDialog.
- Linux: zenity --file-selection.
- macOS: nicht implementiert (Rueckgabe leer).

Rueckgabe:

- Gibt den gewaehlten Pfad auf stdout aus.
- Bei Cancel wird eine **leere Zeile** ausgegeben.

7) Browser schliessen (`close\_browser.py`) - typ. `/closeBrowser``

Ziel: Den Browserprozess finden, der die HTTP-Connection zum lokalen Server aufgebaut hat (Kiosk-Szenario) und ihn beenden.

Core-Funktion:

- `close_browser_from_request(client_ip, client_port, server_ip, server_port, user_agent="", force=False) -> dict`

Wichtige Voraussetzung:

- Der HTTP-Server muss den **Remote-Port** der Verbindung kennen (`client_port`) und den lokalen `server_port`.

Algorithmus (Kurzfassung):

1. PID der Client-Socket-Verbindung bestimmen:

- Windows: `netstat -ano` (primaer: exakte ClientPort->ServerPort Zuordnung), fallback: irgendein Browser, der zu `server_port` verbunden ist.
- Linux: `ss -tnp` (fallback: `/proc` scan ueber inode).

2. Sicherheitscheck: PID muss zu einem bekannten Browser gehoeren (Chrome/Edge/Firefox/Brave/Opera/Chromium). Mit `force=true` kann der Check uebersprungen werden.

3. Root-Prozess bestimmen (Parent-Kette hoch, solange Parent ebenfalls Browser ist).

4. Beenden:

- Windows: `taskkill /T` (und bei Bedarf `/F`).
- Posix: `SIGTERM` und eskaliert zu `SIGKILL`.

Erfolg:

```
{
  "ok": true,
  "platform": "Windows",
  "client_port": 59432,
  "server_port": 8051,
  "user_agent": "Mozilla/...",
  "pid_found": 8888,
  "pid_found_name": "chrome.exe",
  "root_pid": 7777,
  "root_name": "chrome.exe",
  "kill": {"ok": true, "method": "taskkill", "forced": true, "stdout": "...",
    "stderr": ""}
}
```

Fehlercodes:

- `pid_not_found`
- `not_a_browser_process`
- `refuse_kill_self`

Hinweis zu macOS:

- Der Linux-Pfad nutzt `/proc` und `ss`. Auf macOS ist das i.d.R. nicht verfuegbar - in der Praxis ist dieses Modul daher **primär Windows/Linux**.

8) Ordner oeffnen (`open_folder.py`) - typ. `/openFolder``

Ziel: Einen Ordner im nativen Dateimanager oeffnen. Optional wird versucht, das Fenster in den Vordergrund zu holen.

Core-Funktion:

- `openfolder_endpoint(raw_path, create=True, cooldown_sec=2.0, base_dir=<Path>, foreground=False) -> dict`

Parameter:

- `raw_path` - Zielpfad. Wenn Datei: es wird der Parent-Ordner geoeffnet.
- `create (bool)` - wenn Ordner nicht existiert: anlegen.
- `cooldown_sec (float)` - Anti-Spam; innerhalb dieser Zeit wird nicht erneut geoeffnet.
- `base_dir` - Basis fuer relative Pfade.
- `foreground (bool)` - Fokusversuch.

Rueckgabe (geoeffnet):

```
{
  "ok": true,
  "path": "C:/events/event1",
  "created": true,
  "opened": true,
  "skipped": false,
  "platform": "Windows",
  "foreground_requested": true,
  "foreground_ok": false
}
```

Rueckgabe (cooldown aktiv):

```
{
  "ok": true,
  "opened": false,
  "skipped": true,
  "reason": "cooldown",
  "cooldown_sec": 2.0
}
```

OS-Support:

- Windows: `explorer` + optional Fokusheuristik.
- Linux: `xdg-open` + optional `wmctrl/xdotool`.
- macOS: `open` + optional AppleScript `Finder activate`.

9) Drucker (`printer_core.py`) - typ. `/printers/*``

Ziel: Druckerlisten, Default-Drucker setzen, Drucker-GUI oeffnen.

9.1 Drucker auflisten

Core-Funktion:

- `list_printers()` -> dict

Windows:

- PowerShell CIM (`Get-CimInstance Win32_Printer`) liefert Name + Default.

Linux:

- benötigt `lpstat` (CUPS tools).
- optional `lptions` fuer User-Default.

Erfolg:

```
{ "ok": true, "printers": ["PrinterA", "PrinterB"], "defaultPrinter": "PrinterA",  
  "os": "windows" }
```

Fehlercodes:

- `printer_list_failed`
- `lpstat_not_found`
- `lpstat_failed`

9.2 Default setzen

Core-Funktion:

- `set_default_printer(name: str)` -> dict

Erfolg:

```
{ "ok": true, "defaultPrinter": "PrinterB", "verified": true, "os": "windows" }
```

Fehlercodes:

- `no_printer_selected`
- `set_default_failed`
- `lptions_not_found`

9.3 Drucker-GUI oeffnen

Core-Funktion:

- `open_printer_gui(printer: str|None, kind: str)` -> dict

Parameter kind:

- overview (oder leer) - Systemuebersicht.
- pref* - Preferences.
- sonst: Properties.

Windows:

- Overview: control.exe printers
- Preferences/Properties: rundll32 printui.dll,PrintUIEntry.

Linux:

- versucht system-config-printer, gnome-control-center printers, kcmshell*.
- fallback: xdg-open http://localhost:631/printers.

10) Renderer (`render\_core.py`) - typ. `/render/collage` und `/render/fromSession`

Ziel: Eine Collage basierend auf einem Template-XML rendern.

Abhaengigkeiten:

- Python: Pillow (PIL), numpy.

Wichtiger Hinweis:

- next_output_index() scannt das Ausgabeverzeichnis und ist ****nicht parallel-sicher****. Der HTTP-Server sollte Rendering pro Output-Verzeichnis per Lock serialisieren.

10.1 Collage rendern (direkter Payload)

Core-Funktion:

- render_collage_api(payload: dict, base_dir: Path|None = None) -> dict

Pflichtfelder:

- template - Pfad zu template.xml.
- input_dir - Ordner mit Photo_<n>.* und optionalen Assets.
- output_collage - Zielordner fuer gerenderte Collagen.
- output_originals - Zielordner fuer Kopien der Originalfotos.

Optionale Felder:

- prefix (default: collage_)
- ext (default: .png) - .jpg, .jpeg, .png.
- render_config - Pfad zu render_config.json.

- `render_config_inline` - dict mit Overrides (wird deep-merged).

Beispiel-Payload:

```
{
  "template": "C:/booth/templates/current/template.xml",
  "input_dir": "C:/booth/captures/session_001",
  "output_collage": "C:/booth/events/event1/final",
  "output_originals": "C:/booth/events/event1/original_copies",
  "prefix": "collage_",
  "ext": ".jpg",
  "render_config_inline": {
    "greenwall": {"enabled": true, "mode": "auto"}
  }
}
```

Erfolg:

```
{
  "ok": true,
  "index": 1,
  "output_path": "C:/.../final/collage_000001.jpg",
  "output_name": "collage_000001.jpg",
  "greenwall_active": true,
  "greenwall_ref": "C:/.../greenwall.jpg",
  "greenwall_bg": "C:/.../___greenwall.jpg",
  "used_photos": ["C:/.../Photo_1.jpg"],
  "copied_originals": ["collage_000001_Photo_1.jpg"]
}
```

Fehlercodes:

- `missing_params`
- `template_not_found`
- `input_dir_invalid`
- `exception`

10.2 Render aus session.json

Core-Funktion:

- `render_from_session(session_folder: str|Path, base_dir: Path|None = None)`
-> dict

Ablauf:

- Liest `<session_folder>/session.json`.
- Setzt status auf `RENDERING`, rendert, schreibt Ergebnis nach `renderResult`.
- Bei Fehler: `status=ERROR` und `error=<dict>`.

Template-Auflösung:

- bevorzugt `session.render.template`

- sonst Fallback-Suche nach `template.xml` im booth root (z.B. `templates/current/template.xml`, `templates/**/template.xml`, usw.)

Output-Auflösung:

- bevorzugt `session.render.output_collage` und `session.render.output_originals`
- sonst fallback: `eventPath/final` und `eventPath/original_copies` (oder session folder)

11) Empfohlene HTTP-Mappings (Beispiele)

Da `python_server.py` nicht vorliegt, hier ein mögliches, übliches Mapping:

- GET `/service/status?api_key=...&exe=C:/path/ApiServer.exe`
- POST `/service/start` (JSON: { `api_key`, `exe`, ... })
- POST `/service/stop`
- POST `/service/restart`

- GET `/pickUpload?api_key=...&title=Select&path=...&filter=...&subdir=...&prefix=...&overwrite=0`

- POST `/render/collage?api_key=...` (JSON body = `render_collage_api` payload)
- POST `/render/fromSession?api_key=...` (JSON: { `"session_folder"`: "..."})

- GET `/openFolder?api_key=...&path=...&create=1&foreground=0`

- GET `/printers/list?api_key=...`
- POST `/printers/default?api_key=...` (JSON: { `"name"`: "Printer" })
- POST `/printers/gui?api_key=...` (JSON: { `"printer"`: "...", `"kind"`: "overview" })

- POST `/closeBrowser?api_key=...` (Server setzt `client_port` aus der Verbindung; optional `force=true`)

12) Betriebshinweise (Praktisch)

- ****GUI-Aktionen**** (Picker, Explorer, Printer-GUI) müssen auf dem Host laufen, auf dem ein Desktop verfügbar ist.
- ****Cancel ist normal****: Filepicker gibt `ok=false` mit HTTP 200 zurück.
- ****Locks****: Rendering serialisieren (Index-Generierung).
- ****Logging****: Bei Problemen helfen `detail/err` Felder in den Rückgaben.